Tutorial 2 | OS Monsoon'25 Section A

Author → Paarth Goyal

Notes (TAs)

Total Time → ~ 60 minutes

- Bash Scripting
- GDB

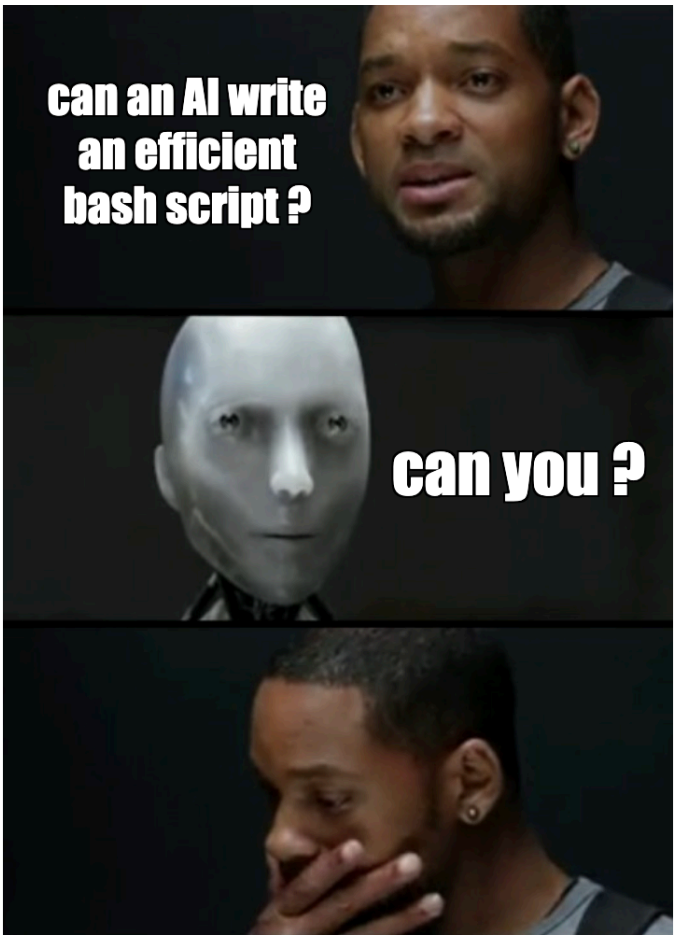
1. Shell Scripting

What is Shell Scripting ?

- Shell scripting is the process of writing a series of commands for the **shell** (command-line interpreter) in a script file, so that they can be executed automatically instead of typing them manually one by one.
- A shell script is basically a text file containing commands that the shell (like **bash**, **zsh**, or **sh**) executes line by line.
- For this tutorial we will utilise bash (Bourne Again Shell).

Why do we use it ?

- **Automate Repetitive Tasks**
- **Native Support on Unix/Linux Systems (No additional installation needed)**
- **Great for Linking Commands to Work Together**
- **Lightweight and Easy to Write (Interpreted and not compiled, hence fast)**



Creating your first script

```
# all shell scripts have a .sh file extension
touch myscript.sh
nano myscript.sh
```

```
#!/bin/bash
echo "User: $USER"
echo "Date: $(date)"
echo "Formatted Date: $(date '+%Y-%m-%d_%H-%M-%S')"
```

`#!/bin/bash` is known as the `shebang` , this can be thought of as a flag which specifies that the bash interpreter is to be used for this script.

Run it →

```
# We won't have the permission to run the script initialy
chmod +x script.sh # gives permission to execute
./script.sh
```

Building Blocks

a. Variables

- Defining a variable

```
#!/bin/bash
name="Paarth"
echo "Greetings, $name!"
```

- Run the script using `./myscript.sh`
- Output :

```
Greetings, Paarth!
```

Note :

- While assigning value to a variable ensure that there are no spaces between variable name, equal sign and variable value.
- The variable value can be accessed using `$` .

Example

```
#!/bin/bash
num1=78
num2=22
sum=$((num1+num2))
echo "The sum is, $sum"
```

- run using `bash myscript.sh` if you don't want to give execute permission first using chmod.

b. User Input

- Use read before variable name to input a value into it at runtime

```
#!/bin/bash
echo "What is your first name ?"
read first_name
echo "What is your last name ?"
read last_name
echo "Hello, $first_name $last_name"
```

- output :

```
What is your first name ?
[user input 1]
What is your second name ?
[user input 2]
Hello, [user input 1] [user input 2]
```

c. Command Line Arguments

```
#!/bin/bash
echo "This is the first argument : $1"
echo "This is the second argument : $2"
```

```
./my_script argument1 argument2
```

- The positional argument at position 0 is reserved for the shell. Any arguments passed can be accessed from `$1` onwards.

d. I/O Redirection

- `>` and `>>` symbols are used to write and append to a file respectively.

```
#!/bin/bash
echo "Hello World!" > hello.txt
echo "Have a wonderful day!" >> hello.txt
cat hello.txt
```

- `>` overwrites the contents of the file with the output of the command preceding it while `>>` simple appends the output of the command preceding it.
- output :

```
Hello World! Have a wonderful day
```

e. Test Operator

- The test operator can be used in order to evaluate an expression to true or false.

```
[ hello = hello ]
# ensure that there is a singular space between every word in the expression
```

- `echo $?` can be used to get the exit code of the last executed command.
- exit code `0` represents that the expression was evaluated to true, exit code `1` represents that the expression was evaluated to false.

f. Conditional Statements

- If, else if and else statements

```
#!/bin/bash
if [ ${1,,} = paarth ]; then echo "Valid user detected. Greetings Paarth!"
elif [ ${1,,} = help ]; then echo "Please enter your username"
else echo "Invalid user detected"
fi
```

- `fi` is used to mark the end of the conditional statement.

g. Arrays

- An array in bash is a variable which stores multiple values of the same data type

```
#!/bin/bash
my_list=(1 2 3 4 5) # elements must be space separated
echo $my_list # outputs only the first element
echo ${my_list[@]} # outputs all elements
echo ${my_list[0]} # outputs element at a specific index
echo ${my_list[4]}
echo ${my_list[27]} # outputs nothing
```

h. Loops

- For loop

```
#!/bin/bash
for i in 1 2 3 4 5; do echo "Number : $i"
done
```

- Iterating through an array using for loop

```
#!/bin/bash
my_list=(one two three four five)
```

```
for i in ${my_list[@]}; do echo -n $i | wc -c;
done
```

- While loop

```
#!/bin/bash
counter=1
while [ $counter -le 5 ]; do echo "Counter : $counter"
((counter++))
done
```

i. Functions

- Functions in bash as in any other programming language allow us to write reusable code chunks which can be called at will.

```
#!/bin/bash
function is_equal(){
    if [ $1 = $2 ]; then echo "Numbers are equal"
    else echo "Numbers are not equal"
    fi
}
is_equal 34 42
is_equal 27 27
```

- Use the positional arguments to pass in values to the function.

j. Error Handling

Example

- Create an empty file using `touch testfile.txt`

```
#!/bin/bash
filename="testfile.txt"
if [ -e $filename ]; then echo "File exists!"
```

```
else echo "File does not exist"
fi
```

- Run the script using `bash my_script.sh`
- remove the file using `rm testfile.txt` and then try to run the script again.

Automating Tasks With Shell Scripting

- example

```
#!/bin/bash
# Ask user for the source directory
echo "Enter the path for the source directory (dir1):"
read dir1

# Ask user for the destination directory
echo "Enter the path for the destination directory (dir2):"
read dir2

# Check if the source directory exists
if [ ! -d "$dir1" ]; then
  echo "Error: Source directory $dir1 does not exist."
  exit 1
fi

# Check if the destination directory exists
if [ ! -d "$dir2" ]; then
  echo "Destination directory $dir2 does not exist. Creating it now."
  mkdir -p "$dir2"
fi

# Move files from source directory to destination directory
mv "$dir1"/* "$dir2"
```

```
# Confirm the move
    echo "Files have been moved from $dir1 to $dir2."
```

2. GDB (GNU Debugger)

A debugger is a program that runs or simulates another program which allows us to :

- Pause and continue its execution
- Set “break points” or conditions where execution is paused so that the programs current state can be observed
- Keep track of values of certain variables
- Go through program execution line by line (or instruction by instruction in case we want to take a look at its assembly code).

Install GDB

```
sudo apt install gdb
```

Know more about GDB

you can refer to GDB’s man page in order to learn about more functionalities that it provides.

```
man gdb
```

Compile and open a C file with GDB

```
gcc -g my_file.c -o my_file
gdb ./my_file
gdb ./myfile arg1 arg2 # if your program accepts any command line arguments
```

Understanding Breakpoints

A breakpoint is a point in the execution of the code post which you can manually navigate through the code one step at a time.

```
(gdb) start
```

the start command is used in order to step through the code line by line starting from the main() function. If we would have called run then the complete program would have executed without giving us the opportunity to examine a specific section in detail.

Basic GDB commands

Command	Function
run	Runs the code until a breakpoint is reached
break POINT	Sets a break point at a specific function or line number
delete n	Removes the break point numbered n
continue	Used to resume execution after program is halted by a breakpoint until the next breakpoint is encountered
info breakpoints	Gives details regarding every breakpoint
step	Move in depth into the current function
next or n	Move to the next line in program execution
print VAR	Print current value of a variable
watch VAR	Halts program execution whenever the value of the variable currently being watched changes
backtrace or bt	Used to get the complete function call stack so that we can know what was the sequence of function calls or where we currently are in the execution sequence
return / finish	Used to return from the current function. Control is passed back to the point where the function was initially called
call function_name()	Used to call a particular function
set VAR = 3	Used to set the current value of a variable
list	Used to display the code of the executable file in the GDB interface itself

layout src (for C code) layout next (for Assembly)	Used to display the complete code in which you can visually observe the current line and execution sequence.

Hands-on with GDB

You can install GDB and use the C file shared with you if you wish to follow along with the demonstration.